

RsCpsi Prime Mode 수정 및 CPSI 비교

1 실험 조건

본 문서는 RsCpsi에 추가한 prime mode 수정 사항과 기존 XOR 기반 CPSI 대비 통신량 및 실행 시간 비교 결과를 정리한다. 비교 실험에서는 sender와 receiver의 식별자 개수를 동일하게 두었다.

$$n_s = n_r = n$$

기존 CPSI와 prime CPSI는 같은 식별자 개수와 같은 associated data 입력 크기를 사용했다.

항목	값	설명
intersection 형태	full intersection	$\forall i, X_i = Y_i$
thread 수	1	nt=1
security parameter	40	mSsp=40
prime	4294967291	2^{32} 보다 작은 큰 prime
prime data 원소 수	1	identifier 하나당 payload 1개
prime data 원소 크기	16 bits	$\ell_p/2$
입력 associated data 크기	2 bytes	1×16 bits
기존 CPSI 모드	Xor	입력 data 크기를 2 bytes로 맞춤
prime CPSI 모드	prime	$\mathbb{Z}_p$ share 사용

통신량은 LocalAsyncSocket::bytesSent()로 측정했다. receiver socket과 sender socket의 송신 byte를 각각 기록한 뒤, 두 값을 더해 전체 통신량으로 사용했다.

2 Prime Mode 수정 사항

2.1 Prime Parameter 및 Length Helper

기존 코드는 prime을 고정 상수로 사용했다.

Listing 1: 이전 코드

```
constexpr u64 RsCpsiPrime = 4294967311ULL;

void init(
    u64 senderSize,
    u64 recverSize,
    u64 valueByteLength,
    u64 statSecParam,
    block seed,
    u64 numThreads,
    ValueShareType type = ValueShareType::Xor);
```

수정 후에는 prime을 init parameter로 전달할 수 있게 하고, 기본값은 2^{32} 보다 작은 큰 prime으로 두었다.

Listing 2: 수정 코드

```
constexpr u64 RsCpsiDefaultPrime = 4294967291ULL;

inline u64 RsCpsiPrimeBitLength(u64 primeModulus)
```

```

{
    return oc::log2ceil(primeModulus);
}

inline u64 RsCpsiDataBitLength(u64 primeModulus)
{
    return RsCpsiPrimeBitLength(primeModulus) / 2;
}

void init(
    u64 senderSize,
    u64 recverSize,
    u64 valueByteLength,
    u64 statSecParam,
    block seed,
    u64 numThreads,
    ValueShareType type = ValueShareType::Xor,
    u64 primeModulus = RsCpsiDefaultPrime);

```

기본 설정에서는

$$\ell_p = 32, \quad L_p = 4, \quad \ell_d = 16, \quad L_d = 2$$

가 된다.

2.2 Input Data와 Share Size 분리

기존 prime mode는 input data와 output share가 모두 u64 단위라고 보고 동일한 byte length를 사용했다.

Listing 3: 이전 코드

```

auto byteLength = type == ValueShareType::prime ?
    4 * sizeof(u64) :
    sizeof(block);

Tv.resize(Y.size() * 3, keyByteLength + values.cols());
ret.mValues.resize(numBins, values.cols());

```

수정 후에는 sender input data 크기와 $\mathbb{Z}_p$ share 크기를 분리했다.

Listing 4: 수정 코드

```

auto dataElemByteLength = RsCpsiDataByteLength(prime);
auto shareElemByteLength = RsCpsiPrimeByteLength(prime);

auto byteLength = type == ValueShareType::prime ?
    dataElemByteLength :
    sizeof(block);

shareByteLength =
    (mValueByteLength / mPrimeDataByteLength) * mPrimeByteLength;

Tv.resize(Y.size() * 3, keyByteLength + shareByteLength);
ret.mValues.resize(numBins, shareByteLength);

```

따라서 기본 prime mode에서 identifier 하나당 input associated data는 $1 \times 16 = 16$ bits이고, output share는 $1 \times 32 = 32$ bits이다.

2.3 Prime Element Read/Write 가변화

이전 코드는 prime 원소를 항상 8 bytes로 읽고 썼다.

Listing 5: 이전 코드

```

u64 readU64(const u8* src)
{

```

```

    auto v = u64{};
    std::memcpy(&v, src, sizeof(v));
    return v;
}

void writeU64(u8* dst, u64 v)
{
    std::memcpy(dst, &v, sizeof(v));
}

```

수정 후에는 필요한 byte 수만 읽고 쓰도록 변경했다. 계산 자료형은 u64이지만 통신 payload는 byteLength만큼만 사용한다.

Listing 6: 수정 코드

```

u64 readPrimeElement(const u8* src, u64 byteLength)
{
    auto v = u64{};
    std::memcpy(&v, src, byteLength);
    return v;
}

void writePrimeElement(u8* dst, u64 byteLength, u64 v)
{
    std::memcpy(dst, &v, byteLength);
}

```

2.4 검증 로직 제거

Listing 7: 이전 코드

```

if (mType == ValueShareType::prime)
{
    validatePrimeValues(values, mPrimeDataBitLength, mPrimeDataByteLength);
    shareByteLength = primeShareByteLength(...);
}

u64 primeShareByteLength(u64 valueByteLength, u64 dataByteLength, u64 primeByteLength)
{
    if (dataByteLength == 0 || valueByteLength % dataByteLength)
        throw RTE_LOC;

    return (valueByteLength / dataByteLength) * primeByteLength;
}

assert(values.cols() % mPrimeDataByteLength == 0);

if (tv >= mPrime || rr >= mPrime)
{
    co_await chl.close();
    throw RTE_LOC;
}

```

수정 후에는 input value 범위 검증, element size 나눠떨어짐 검증, prime element별 p 이상 여부 검증을 수행하지 않고 share 계산을 진행한다.

Listing 8: 수정 코드

```

if (mType == ValueShareType::prime)
{
    shareByteLength = primeShareByteLength(...);
}

u64 primeShareByteLength(u64 valueByteLength, u64 dataByteLength, u64 primeByteLength)
{

```

```

    return (valueByteLength / dataByteLength) * primeByteLength;
}

writePrimeElement(
    &*TvIter + dstOffset,
    mPrimeByteLength,
    modSubPrime(tv, rr, mPrime));

```

2.5 통신량 및 시간 측정 방법

기존 CPSI와 prime CPSI는 같은 n , 같은 input associated data size 조건에서 비교했다. 프로토콜 코드는 prime mode 구현에 필요한 부분 외에는 수정하지 않았고, 실험 중에는 로컬 비동기 소켓의 `bytesSent()` 값을 읽어 receiver/sender 송신량을 기록했다.

시간은 단일 실행값 대신 warm-up 이후 여러 번 반복 실행한 평균으로 정리했다. 실행 순서에 따른 cache 및 allocator 영향을 줄이기 위해 XOR와 Prime의 실행 순서를 번갈아 측정했다. 통신량 표에는 실제 socket 송신량만 기록했다.

3 이론적 통신량

이론식은 논문에서 주로 사용하는 dominant communication term 기준으로, 전체 CPSI 프로토콜의 주요 통신 항을 근사한다. 이번 비교에서는 $n_s = n_r = n$ 이고, cuckoo table 크기는 약 $1.3n$ 으로 둔다. n 은 identifier/data 개수, ℓ 은 identifier 하나당 payload bit 길이이다. 따라서 XOR mode는 $\ell = 16$, prime mode는 $\ell = 32$ 이다.

전체 이론 통신량은 다음 세 항의 합으로 둔다.

$$C_{\text{total}}(n, \ell) \approx C_{\text{OPRF}}(n) + C_{\text{OPPRF}}(n, \ell) + C_{\text{PEQT}}(n)$$

각 항은 다음처럼 근사한다.

$$C_{\text{OPRF}}(n) = 128 \cdot 1.3n$$

$$C_{\text{OPPRF}}(n, \ell) = 3 \cdot 1.3n \cdot (40 + \lceil \log_2 n \rceil + \ell)$$

$$C_{\text{PEQT}}(n) = 4 \cdot 1.3n \cdot (40 + \lceil \log_2 n \rceil)$$

여기서 OPPRF의 계수 3은 sender 원소 하나가 hash 위치 3개를 만들기 때문이다. PEQT는 약 $1.3n$ 개의 cuckoo bin에서 $40 + \lceil \log_2 n \rceil$ bit equality를 확인한다고 본다. mode 간 이론적 차이는 ℓ 차이에서 생기므로

$$\Delta(n) \approx 3 \cdot 1.3n \cdot (32 - 16)$$

bits이다. 실제 구현에는 OT/VOLE 내부 metadata, packing, Baxos rounding 등이 포함되므로 이론 값과 실측값이 완전히 일치하지는 않는다.

4 실측 통신량 및 시간 비교

아래 결과는 $n \in \{2^{14}, 2^{16}, 2^{18}, 2^{20}\}$ 에 대해 warm-up 1회 이후 10회 반복 실행한 평균값이다. 실행 순서에 따른 캐시/allocator 영향을 줄이기 위해 round마다 XOR/Prime 실행 순서를 번갈아 사용했다. 실험은 병렬로 실행하지 않고, 아래 명령을 n 값만 바꿔가며 순차적으로 실행했다.

```

out/build/osx/frontend/frontend \
-u Cpsi_Rs_comm_time_compare_test \
-n <n> -rounds 10 -warmup 1

```

여기서 `-rounds 10`은 측정 round 10회를 의미하고, `-warmup 1`은 측정 전에 XOR/Prime을 한 번씩 먼저 실행한다는 뜻이다. 실험에 사용한 n 은 $2^{14}, 2^{16}, 2^{18}, 2^{20}$ 이다.

n	mode	theory total bits	actual total bits	mean time(s)
2^{14}	XOR	11799757	24046728	1.731481
2^{14}	Prime	12822118	25100760	1.729537
2^{16}	XOR	48391782	73614560	4.350970
2^{16}	Prime	52481229	77883104	4.493483
2^{18}	XOR	198338150	283386208	15.518204
2^{18}	Prime	214695936	300497248	15.486014
2^{20}	XOR	812436685	1143349664	93.180823
2^{20}	Prime	877867827	1212015008	73.263364

actual total bits는 receiver와 sender의 `bytesSent()`를 더한 실제 통신량이다. theory total bits는 3절의 전체 프로토콜 근사식으로 계산한 값이다. mean time은 warm-up 이후 반복 실행한 평균 시간이다.

5 차이 분석

n	actual extra bits	theory extra bits	comm 증가율	time 차이(s)
2^{14}	1054032	1022362	4.38%	-0.001944
2^{16}	4268544	4089446	5.80%	0.142513
2^{18}	17111040	16357786	6.04%	-0.032190
2^{20}	68665344	65431142	6.01%	-19.917459

실측 통신량 증가분은 이론식의 extra bits와 비슷한 크기로 나타난다. 차이가 나는 주된 이유는 이론식이 $1.3n$ 또는 $1.3 \cdot 3n$ 형태의 근사값을 사용하지만, 실제 구현에서는 byte packing, Baxos/Paxos 내부 rounding, metadata 등이 추가되기 때문이다. 통신량이 증가하는 핵심 이유는 OPRF correction에 들어가는 value payload가 XOR mode에서는 16-bit data share인데, prime mode에서는 32-bit $\mathbb{Z}_p$ share가 되기 때문이다.

$$\ell_v^{\text{prime}} - \ell_v^{\text{xor}} = 32 - 16 = 16$$

따라서 prime mode의 통신량 증가는 명확하다. 반면 end-to-end 평균 시간은 모든 n 에서 단조롭게 증가하지 않았다. 시간 측정값은 prime 산술만이 아니라 OPRF/OPPRF/Paxos/GMW, memory allocation, packing 및 scheduling 영향을 함께 포함하기 때문에, 통신량 증가와 실행 시간 증가가 항상 같은 방향으로 나타나지는 않는다.

6 테스트 결과

다음 명령으로 CPSI 관련 테스트를 실행했고 모두 통과했다.

```
out/build/osx/frontend/frontend -u Cpsi_Rs
```

추가로 비교 실험을 다음 크기들에 대해 실행했다.

$$n \in \{2^{14}, 2^{16}, 2^{18}, 2^{20}\}$$

각 실행에서 XOR mode와 prime mode 모두 full intersection size가 n 임을 확인했다.

CPSI Part 별 통신량 비교

본 절에서는 $n = 2^{10} = 1024$ 일 때 CPSI를 실제로 한 번 실행하여, OPRF, OPPRF, PEQT, Other part에서 발생한 통신량을 각각 측정하였다. 측정은 전체 CPSI 실행 이후 차이를 빼는 방식이 아니라, 실제 프로토콜 실행 중 각 part의 송신 구간을 직접 계측하는 방식으로 수행하였다. 따라서 아래 표의 실측값 합은 전체 CPSI 실측 통신량과 일치한다.

이론식

이론적 통신량은 다음 근사식을 사용하였다. 여기서 n 은 데이터 개수, ℓ 은 payload 크기(bit)를 의미한다. 본 실험에서는 XOR mode의 payload를 16 bit, prime mode의 payload를 32 bit로 두었다.

$$C_{\text{OPRF}} = 128 \cdot 1.3n$$

$$C_{\text{OPPRF}} = 3 \cdot 1.3n \cdot (40 + \log_2 n + \ell)$$

$$C_{\text{PEQT}} = 4 \cdot 1.3n \cdot (40 + \log_2 n)$$

Other part의 이론값은 cuckoo seed 전송에 해당하는 128 bit로 두었다. 반면 실측값은 coproto message framing 등 실제 실행 시 발생하는 통신량을 bit 단위로 환산한 값이다.

Table 1: Part 별 이론 통신량과 XOR/prime mode 실측 통신량 비교 ($n = 2^{10}$, $m = 1305$, $40 + \log_2 m = 51$ bit)

Part	XOR 이론 (bit)	prime 이론 (bit)	XOR 실측 (bit)	prime 실측 (bit)
OPRF	170,393.6	170,393.6	4,954,712.0	4,954,712.0
OPPRF	267,571.2	331,468.8	299,032.0	365,384.0
PEQT	271,564.8	271,564.8	1,517,448.0	1,517,448.0
Other	128.0	128.0	384.0	384.0
Total	709,657.6	773,555.2	6,771,576.0	6,837,928.0

표에서 prime mode의 증가분은 주로 OPPRF value에 포함되는 payload 길이가 16 bit에서 32 bit로 증가한 데서 발생한다. OPRF와 PEQT는 payload 크기에 직접 의존하지 않으므로 두 mode에서 동일한 값으로 측정되었다.